Session-Scroll & History — Multi-Agent-Analyse

AgenticExplorer v1.6.26 — post-release Deep-Dive

4-Agent-Team · Bug-Fix-Pipeline Phase 1 · 2026-04-24

> User-Pain: „Ständige Probleme mit Claude-Session-History-Scrollen" — trotz v1.6.25 + v1.6.26 Patches

Problem-Statement

Symptome (User-Beschwerden über mehrere Releases):

1. Viewport springt unerwartet bei neuem Output
2. Scrollback wirkt abgeschnitten / geht bei Layout-Switches verloren
3. Auto-Scroll-to-Bottom greift manchmal nicht wie erwartet
4. Ctrl+C / Ctrl+R-Verhalten inkonsistent
5. History „verschwindet" bei langen Sessions

Frühere Patches:

Commit	Release	Strategie
`516eff3`	v1.5.x	150ms `scrollTrackTimer` + Rust-Event-Dedup
`8b820f5`	v1.6.25	Always-Mounted (keine Remount bei Tab-Switch)
`d12f0ec`	v1.6.25	Unified CSS-Grid-Tree (keine Ternary bei Layout-Switch)
`176c78b`	v1.6.26	Async unlisten Error-Logging
`f15b039`	v1.6.26	Release

→ User sagt: **„ständig"** → Root Cause nicht getroffen.

Agent-Team (Phase 1)

Agent	Layer	Fokus
D1	Backend (Rust)	`portable-pty` `manager.rs` UTF-8, ConPTY, Reader-Thread
D2	Frontend (xterm-Pipeline)	`SessionTerminal.tsx` `SessionManagerView.tsx` Event-Flow, Races
D3	Config + Claude-TUI	xterm-Optionen, Claude-Ink-Rendering, Alt-Screen, `windowsPty`
D4	Git-Archäologie	Commit-Timeline, Patches-vs-Root-Causes, Test-Coverage-Lücken

Jeder Agent: Code-Zitate mit Zeilennummern, ranked Hypothesen, Anti-Hypothesen, offene Fragen.

Ultrathink. Parallel dispatched.

D1 — Rust/PTY · Kernbefund

UTF-8-Split am 4096-Byte-Buffer-Rand

`manager.rs:179–190`:

```
let mut buf = [0u8; 4096];
// ...
Ok(n) => {
    let data = String::from_utf8_lossy(&buf[..n]).to_string();
```

Mechanismus:

- Emoji (😀 = 4 Bytes), Box-Drawing (`─│┘`), Braille-Spinner (`⠹`) sind Multi-Byte
- Wenn letzter Codepoint exakt am Byte 4095 beginnt: `from_utf8_lossy` → `U+FFFD` (` `)
- Die fehlenden Tail-Bytes sind **verloren**
- Beim nächsten Read starten sie als Header → wieder invalid → doppelter Datenverlust

Confidence D1: 95%

D1 — ANSI-Escape-Fragment-Split

Gleiche Zeile, zweite Bombe:

ANSI-Sequenzen wie `\x1b[38;5;214m` können am Buffer-Ende gesplittet werden:

- Chunk 1 endet mit `\x1b[38;5;2`
- xterm-js-Parser erwartet komplette SGR-Sequenz
- Incomplete Sequences **desynchronisieren den State-Machine**
- Nächste Zeichen werden als Parameter interpretiert statt als Text

Direkte Scroll-Folge:
Wenn ein `CSI <N> A` (Cursor-Up) oder `CSI ? 1049 h/l` (Alt-Screen) beschädigt ist, rendert xterm in falsches Row/Buffer → **Viewport-Jump**.

Confidence: **95%** — mechanistisch erzwungen bei hoher Output-Rate.

D1 — Reader-Thread-Backpressure

`manager.rs:240-285` — Reader-Loop ist synchron:

```
Ok(n) => {
    // ...emit session-output event...
    let stripped = Self::strip_ansi(&data);          // ~0.5-2ms char-by-char
    let agent_events = agent_detector.feed(&stripped); // Regex + Buffer-Ops
    for event in agent_events { /* emit */ }
}
```

Bei Claude-Spinner-Burst (5–10 Events/sec) blockiert der Reader 2–20 ms pro Chunk.
Währenddessen:

- PTY `read()` ist nicht aktiv
- ConPTY baut Backpressure
- Claude-Child pausiert

Scroll-Folge: Frontend-Timer (z. B. 50ms für `fit()`) glaubt Session sei idle → scrollt zu Bottom → dann kommt Burst → **Viewport-Jump**.

Confidence: **85%**

D1 — Initial PTY 24×80 Race

`manager.rs:92-98`:

```
let pty_pair = pty_system.openpty(PtySize {
    rows: 24, cols: 80, pixel_width: 0, pixel_height: 0,
})?;
```

Zeitachse:

1. PTY öffnet mit 24×80 (hardcoded)
2. Reader-Thread startet sofort (`manager.rs:177`)
3. Claude produziert Welcome/Banner/Early-Output bei 24×80
4. Frontend mountet (~100–300 ms), FitAddon berechnet echte Größe (z. B. 40×120)
5. `resize_session` Command an Rust
6. **Aber:** Ink hat bereits Layout-Cache + Wrap-Punkte bei 80 cols

Folge: Zeilen wrappen falsch, werden sichtbar durcheinander geworfen bei nachträglichem Reflow.

Confidence: **78%**

D1 — Weitere Findings

Nr	Befund	Zeilen	Confidence
D1-5	`assert!()` in `shell_args` für `resume_session_id` → Panic in Release wegen `panic="abort"`	`manager.rs:436-443`	90%
D1-6	Inkonsistente Mutex-Poison-Handhabung: `resize_session` macht `into_inner()` `close_session` macht `map_err`	`manager.rs:373` vs `395`	80%
D1-7	`strip_ansi` ignoriert OSC-Sequenzen `\x1b]...BEL` — Cursor-Change/Title-Events bleiben im Feed	`manager.rs:457-477`	70%
D1-8	`detect_status` auf 200-Char-Snippet → Spinner-Context verfehlt	`manager.rs:204-215`	60%
D1-9	Agent-Detector `MAX_BUFFER=4000` + Char-Boundary-Scan — kann bei Burst droppen	`agent_detector/state.rs:59-95`	65%

D2 — Frontend · Kernbefund

Stale `isAtBottom` im `term.write`-Callback

`SessionTerminal.tsx:138–143`:

```
term.write(event.payload.data, () => {
  if (!userScrolledUpRef.current) {
    term.scrollToBottom();
  }
});
```

Race-Condition:

- xterm-js `write()` ist asynchron, Callback feuert nach Render
- Bei schnellem Stream queuen sich 50+ Callbacks
- Callback-Check ist Snapshot **zu spät**: User hat zwischen Write #1 und #50 gescrollt
- Späte Callbacks feuern weiterhin mit `scrollToBottom()`

Effekt: User scrollt hoch → 100ms später zieht späte Callback-Kaskade das Viewport wieder nach unten → Gefühl: „springt hin und her".

Confidence D2: **75%**

D2 — Resize-Debounce-Gap

`SessionTerminal.tsx:165-173` — 100ms Debounce auf `fit()`:

```
const debouncedFit = debounce(() => { runFit(); }, 100);
const resizeObserver = new ResizeObserver(() => { debouncedFit(); });
resizeObserver.observe(containerRef.current);
```

Race-Fenster:

```
T=0ms   ResizeObserver fires → debouncedFit queued
T=50ms  session-output arrives → term.write(data) mit ALTEN dims
T=100ms debouncedFit fires → fit() → new cols/rows
        → xterm reflows, Zeilen wrappen anders
        → viewport-Position ändert sich
T=150ms write-callback from T=50ms feuert
        → isAtBottom() mit NEUER Geometrie
        → scrollToBottom() oder nicht — je nach Race
```

Folge: Visueller „Jump" mit scheinbar verlorener Historie.

Confidence: **60%**

D2 — Event-Flow (ASCII)

```
[Rust PTY Reader]
  ↓ 4096 bytes, from_utf8_lossy  ← (D1: UTF-8/ANSI Split!)
[session-output event]
  ↓ Tauri event queue (no backpressure, no ordering guarantee)
[SessionTerminal.tsx listen callback]
  ↓ term.write(data, callback)
[xterm write-queue]                ← (D2: async, multiple pending cbs)
  ↓ render (DOM renderer, no WebGL)
[callback fires LATER]
  ↓ userScrolledUpRef check (STALE!)
  ↓ scrollToBottom() maybe
[USER SEES]
  ↓ jump / no jump / ping-pong

  [ResizeObserver] —debounce 100ms—▶ [fit()] —▶ [resize_session RPC] —▶ [Rust resize PTY]
              ↑ concurrent with stream above
```

Race-Zonen: **term.write ↔ fit(), write-cb ↔ onScroll, initial fit ↔ font-ready**.

D2 — Always-Mounted: Gut, aber nicht Selbstverständlich

`SessionManagerView.tsx` (post v1.6.25/1.6.26):

- Alle Sessions gemountet, Sichtbarkeit via `display:none` / `display:flex`
- `SessionTerminal` useEffect-Dependency ist `[sessionId, isAtBottom]` — **keine Layout-Mode-Dep**
- `term.dispose()` feuert nicht bei Tab/Layout-Switch

Aber: Neue Race bei `display:none → display:flex`:

- ResizeObserver feuert sofort (0×0 → echte Größe)
- 100ms Debounce läuft
- In dem Fenster: `offsetWidth > 0` aber `fit()` noch nicht durch
- Output-Chunks schreiben mit alter Geometrie

Confidence: **55%**

D2 — Weitere Findings

Nr	Befund	Zeilen	Confidence
D2-4	`document.fonts.ready.then(runFit)` — Promise settles nie offline/fehlende Font → `resize_session` wird nie gesendet	`SessionTerminal.tsx:176-180`	35%
D2-5	`SCROLL_BOTTOM_THRESHOLD = 1` zu aggressiv bei Jitter	`SessionTerminal.tsx:15`	45%
D2-6	`scrollTrackTimer` 150ms könnte post-d12f0ec obsolet sein	`SessionTerminal.tsx:112-121`	50% (open question)
D2-7	`onData` → `wrapInvoke("write_session")` Error-Pfad lässt getippte Zeichen im Buffer hängen	`SessionTerminal.tsx:124-128`	40%
D2-8	Ctrl+R (PSReadLine reverse history) kein Custom-Handler → default xterm-Verhalten	`SessionTerminal.tsx:93-107`	offene Frage

D3 — xterm-Config · Konfiguration Review

Option	Aktuell	Empfohlen	Warum
`scrollback`	5000	5000	OK, aber Layer-4-Bug macht's wirkungslos
`allowProposedApi`	`true`	`true`	Nötig für `buffer.active.*`
`scrollOnUserInput`	(default `true`)	**`false`**	Race mit `userScrolledUpRef`
`windowsPty`	(nicht gesetzt)	`{ backend: 'conpty', buildNumber: 19041 }`	CRITICAL: ConPTY-Wrap-Detection
`convertEol`	(nicht gesetzt)	`true`	Normalisiert bare `\n` von Node-Kindprozessen
`smoothScrollDuration`	(default 0)	`0`	OK — explicit halten
WebGL-Renderer	nicht geladen	`@xterm/addon-webgl` optional	Performance, keine Scroll-Fix

D3 — Claude-TUI Rendering-Modell

Claude-CLI = **Ink** (React für CLI). Rendering pro Update:

```
CSI ? 1049 h   ← Alt Screen ON (beim TUI-Start)
CSI 2J         ← Clear Screen
CSI H          ← Home Cursor
[Text rewrite]
CSI <N> A      ← Cursor up N zeilen (für nächstes eraseLines)
```

Das Problem:
xterm.js pusht Zeilen in Scrollback **nur bei `LF` oder CUD mit Wrap-Marker**.
Claudes `eraseLines + rewrite` nutzt `CSI <N> A` + Overwrite → **Zeilen landen nie im Scrollback**.

→ Egal ob `scrollback: 5000` oder `50000` — **die alten Zeilen sind nie dort gewesen**, nur im Alt-Screen-Viewport.

Confidence D3: **85%** — dies ist wahrscheinlich die zentrale Erklärung.

D3 — Issue #41965: CLAUDE_CODE_NO_FLICKER

Claude-Code v2.1.89+: „flicker-free rendering" mit virtualisiertem Scrollback Default AN.

Bei Output > viewport-height (~24 Zeilen) → **gesamter Screen wird neu re-rendered**:

- Banner/Maskottchen erscheint mehrfach
- Zwischen-Output verschwindet
- Nur ~2 Bildschirmseiten Scrollback übrig

Explizit bestätigt für xterm.js + Tauri WebView2 (unser Setup).

Escape-Hatch: Env-Variable `CLAUDE_CODE_NO_FLICKER=0`

→ Claude fällt zurück auf v2.1.87-Verhalten: linear, `\n`-basiert, Scrollback funktioniert.

Confidence: **90%** — direkt aus Issue-Report verifiziert.

D3 — Issue #37389: eraseLines Overshoot

Bei ~150k+ Tokens + 60+ Tool-Calls:

Ink kalkuliert `eraseLines(N)` basierend auf akkumuliertem History-State.
N wächst über Zeit → `CSI <N> A` schiebt Cursor **in den Scrollback-Bereich oberhalb des Viewports**.
xterm-js folgt → **Viewport snapt to top**.

Für xterm.js 5.5 in Tauri-WebView2 **explizit bestätigt**. Für v6 ungetestet, aber mechanistisch identisch.

Kein offizieller Fix. Anthropic hat Duplicate-Issues mit 6 anderen Reports.

Workaround-Wirkung von `CLAUDE_CODE_NO_FLICKER=0`: adressiert symptomatisch, weil der Re-Render-Pfad deaktiviert wird.

D3 — Alt-Screen-Buffer vs. Scrollback

Schlüsselfrage: Nutzt Claude Alt-Screen?

- `CSI ? 1049 h` beim TUI-Start → Scrollback ist **pro Buffer**
- User scrollt im Alt-Screen → sieht nur das was Ink gerade hält
- Hauptbuffer-Historie (Welcome/Pre-Prompt) ist im Normal-Buffer → nicht sichtbar vom User während Claude läuft

Interaktion mit unserer `scrollToBottom`-Logik:
Wenn Ink `CSI ? 1049 l` (Exit Alt-Screen) macht (z. B. beim `claude`-Exit oder `$EDITOR`-Spawn), springt xterm zurück auf Normal-Buffer → **großer visueller Jump**.

Fix: `@xterm/addon-serialize` könnte State snapshotten.

Confidence: **60%**

D3 — Font-Reflow-Race

`SessionTerminal.tsx:79` + `176-180`:

```
term.open(containerRef.current);       // fit wird mit Fallback-Font berechnet
// ...150ms später:
document.fonts.ready.then(() => { runFit(); });  // fit mit Cascadia-Breite
```

Folge:

- xterm's `charWidth` wird beim ersten `open()` gecached
- Beim späteren `fit()` ändert sich charWidth (Cascadia ≠ Consolas)
- xterm **reflowed den kompletten Buffer** mit neuer Breite
- Wrap-Punkte verschieben sich → Zeilen rutschen im Viewport

Confidence: 85% (D3)

Fix: `term.open()` erst NACH `document.fonts.ready`.

D3 — Testbare Config-Matrix

#	Experiment	Erwartung	Aufwand
T-C1	`windowsPty: { backend: 'conpty', buildNumber: 19041 }`	Wrap-Detect korrekt bei Resize	5 min
T-C2	`scrollOnUserInput: false` + manuell	Auto-Scroll-Race weg	10 min
T-C3	`convertEol: true`	Bare `\n` von Node-Children normalized	2 min
T-C4	`term.open()` nach `fonts.ready`	Kein Reflow-Jump nach Font-Load	10 min
T-C5	`@xterm/addon-serialize` bei Alt-Screen-Exit	Cross-Buffer-State erhalten	30 min
T-C6	`@xterm/addon-webgl`	Reines Rendering-Perf, kein Scroll-Fix	5 min

D4 — Git-Archäologie · Historischer Pfad

Commit	Kat.	Was es löste
`9e769b0` / `78b4553`	Symptom	CSS `min-h-0` in Flex-Chain — Sidebar-Scroll
`516eff3`	Root (teilweise)	Rust-Event-Dedup (100–200x/s → ~5x/s); **Symptom:** 150ms `scrollTrackTimer`
`06656e9`	Root	Custom Ctrl+V Handler entfernt (Double-Paste-Bug)
`8b820f5`	**Root**	Always-Mounted Strategie — kein Remount bei Tab-Switch
`c8d64d3`	Test	Regression-Test (Unmount-Spy)
`d12f0ec`	**Root**	Unified CSS-Grid-Tree — kein Remount bei Layout-Switch
`176c78b`	Diagnose	Async unlisten Error-Logging
`f15b039`	Release	v1.6.26 Aggregator

→ Historische Remount-Defekte sind **architektonisch gelöst**.

D4 — Patching-Muster-Diagnose

Symptom-Patching (alert sign):

- `516eff3` 150ms `scrollTrackTimer` — empirischer Timing-Workaround
- Drei Commits in 10 Tagen verdrahten `userScrolledUpRef` → Symptom-Suche

Root-Cause-Fixes:

- `8b820f5` + `d12f0ec` eliminieren **React-Mount-Anti-Pattern** (Ternary → Remount)
- Das ist **architekturell** gelöst, nicht mit Guard

Offene Frage post-d12f0ec:

- 150ms `scrollTrackTimer` war wahrscheinlich Symptom-Fix für Event-Spam
- Nach Event-Dedup (Rust) + Always-Mount (Frontend) **potentiell obsolet**
- Experiment T-G1: entfernen, Regression-Tests laufen lassen, A/B-Vergleich

D4 — Test-Coverage-Lücken

Getestet:

- Scrollback 1000+ Zeilen konfiguriert
- Auto-Scroll bei Bottom
- Auto-Scroll stoppt wenn User hochgescrollt
- Ctrl+C Copy/SIGINT
- Unmount-Spy bei Tab/Layout-Switch

NICHT getestet:

- Font-Ready-Race (charWidth-Reflow)
- `term.write`-Callback-Queue-Backlog bei Burst
- UTF-8-Split am 4096-Byte-Rand
- ANSI-Fragment-Corruption
- Alt-Screen Enter/Exit
- `scrollTrackTimer` 150ms: notwendig oder redundant?
- Ctrl+R (PSReadLine reverse search)

Kritisch: Keine End-to-End-Tests mit echter Claude-Session unter Burst-Load.

SYNTHESE — Layered Defect Map

```
┌──────────────────────────────────────────────────────────────┐
│ LAYER 6 — Historische Architektur (GELÖST in v1.6.26)        │
│ ✓ Always-Mount    ✓ Unified Grid-Tree                        │
├──────────────────────────────────────────────────────────────┤
│ LAYER 5 — xterm-Config (OFFEN)                               │
│ ✗ windowsPty fehlt    ✗ scrollOnUserInput default-true       │
├──────────────────────────────────────────────────────────────┤
│ LAYER 4 — Claude-TUI-Protokoll (EXTERN, OFFEN)               │
│ ✗ eraseLines statt LF    ✗ Alt-Screen    ✗ #41965/#37389     │
├──────────────────────────────────────────────────────────────┤
│ LAYER 3 — xterm Write-Pipeline (OFFEN)                       │
│ ✗ stale isAtBottom    ✗ Resize-Debounce-Gap    ✗ Font-Reflow │
├──────────────────────────────────────────────────────────────┤
│ LAYER 2 — Tauri-Event-Queue (LOW)                            │
│ ~ keine Backpressure    ~ spekulatives Reordering            │
├──────────────────────────────────────────────────────────────┤
│ LAYER 1 — PTY-Byte-Stream (KRITISCH, OFFEN)                  │
│ ✗ UTF-8-Split    ✗ ANSI-Fragment    ✗ Reader-Backpressure    │
└──────────────────────────────────────────────────────────────┘
```

SYNTHESE — Kern-Einsicht

> **Der Remount-Bug ist gelöst. Die „ständigen Probleme" kommen aus einer Schichten-Kaskade die sich nicht mit einem einzelnen Fix lösen lässt — weil Claudes Ink-Renderer fundamentale Annahmen von xterm.js verletzt.**

Die härteste Wahrheit:

- xterm.js geht davon aus: alte Zeilen gehen via `LF` in den Scrollback.
- Ink geht davon aus: alte Zeilen werden via `CSI A` + rewrite **überschrieben**.
- Kein `scrollback: 50000` hilft, wenn die Zeilen nie dorthin geschickt werden.

Konsequenz:
„History sichtbar erhalten" erfordert **entweder**:

1. Claude zur LF-Mode zwingen (`CLAUDE_CODE_NO_FLICKER=0`), ODER
2. Externe State-Serialisierung (`@xterm/addon-serialize` an Alt-Screen-Boundaries), ODER
3. Akzeptieren, dass die TUI-Historie flüchtig ist, und parallel eine eigene Session-Log-Persistenz bauen.

RANKED HYPOTHESEN — Konsolidiert

#	Hypothese	Confidence	Layer	Impact
H1	UTF-8/ANSI-Split am 4096-Byte-Rand korruptiert xterm-State	95%	1	hoch
H2	Claudes `eraseLines`+rewrite schickt Zeilen nie in Scrollback	85%	4	**sehr hoch**
H3	Font-Reflow nach `document.fonts.ready` triggert Buffer-Reflow	85%	3	mittel
H4	Reader-Thread-Backpressure durch synchronen `agent_detector`	85%	1	mittel
H5	Stale `isAtBottom` in asynchronen `term.write`-Callbacks	75%	3	hoch
H6	`scrollOnUserInput: true` konkurriert mit `userScrolledUpRef`	70%	5	mittel
H7	Initial 24×80 PTY-Race produziert falsch-gewrappten Early-Output	78%	1/4	mittel
H8	Fehlendes `windowsPty` → ConPTY-Wrap-Detection falsch	55%	5	mittel
H9	Resize-Debounce-Gap: Output schreibt mit alter Geometrie	60%	3	mittel
H10	Alt-Screen-Enter/Exit verursacht großen Viewport-Jump	60%	4	niedrig

ANTI-HYPOTHESEN

Was es **nicht** ist:

- ❌ xterm-Buffer wird bei `display:none` gelöscht — nein, Buffer ist DOM-unabhängig
- ❌ Scroll-Tracking-Timer 150ms zu kurz — nein, war ursprünglich empirisch passend
- ❌ `attachCustomKeyEventHandler` für Ctrl+C stört Scroll — nein, greift nicht in xterm-State ein
- ❌ Pure xterm-Rendering-Bug — nein, xterm ist robust **bei korrektem Input**
- ❌ React-Rerender triggert Scroll — nein, Scrollback-Loss passiert in xterm-Buffer, nicht JSX
- ❌ ConPTY-Encoding-Bug — nein, ConPTY ist UTF-8-korrekt; Bug ist Rust-seitig
- ❌ Agent-Detector-Crash — nein, aber er **verlangsamt** den Reader synchron

TEST-MATRIX — Tier 1 (Cheap, High-Value)

#	Experiment	Datei / Ort	Test-Szenario	Erwartung	Aufwand
T1	`cmd.env("CLAUDE_CODE_NO_FLICKER", "0")` beim Spawn	`manager.rs:107`	Lange Session, lange Outputs, Scrollen hoch	Banner nur 1× sichtbar, Scrollback linear, `<N>A` Overshoot weg	15 min
T2	`windowsPty: { backend: 'conpty', buildNumber: 19041 }`	`SessionTerminal.tsx:62`	Fenster-Resize, Text selectieren der wrappt, kopieren	Copy-Text enthält korrekte Wraps	5 min
T3	`scrollOnUserInput: false` + manual im `onData`	`SessionTerminal.tsx:62` + `:124`	Hochscrollen, tippen	Viewport bleibt oben solange User nicht manuell bottom-scrollt	15 min
T4	UTF-8-Carry-Buffer im Rust Reader	`manager.rs:179-195`	Claude mit Emoji-Ausgabe, langem Output	Keine `⁉` mehr, keine nackten `[38;5;2` im Terminal	90 min

TEST-MATRIX — Tier 2 (Medium)

#	Experiment	Ort	Szenario	Aufwand
T5	`term.open()` nach `document.fonts.ready` (umkehren der Reihenfolge)	`SessionTerminal.tsx:79 + :176`	Font-DevTools-Throttle, Terminal öffnen	30 min
T6	Suppress `scrollToBottom` während 200ms Resize-Fenster	`SessionTerminal.tsx:138 + :165`	Schnelles Fenster-Resizing während Stream	45 min
T7	`@xterm/addon-serialize` an Alt-Screen-Boundary	`SessionTerminal.tsx`	Claude starten, `$EDITOR` spawnen, verlassen	90 min
T8	`convertEol: true`	`SessionTerminal.tsx:62`	Node-Kinds mit bare `\n`	5 min

TEST-MATRIX — Tier 3 (Architektur)

#	Experiment	Ort	Aufwand	Risiko
T9	Async `agent_detector` via mpsc-Channel	`manager.rs:177–294`	3 h	mittel — muss Ordering wahren
T10	Dynamische Initial-PTY-Size (Frontend liefert Default)	`commands.rs` + `manager.rs:92`	2 h	niedrig
T11	`scrollTrackTimer` 150ms entfernen + Regression-Test	`SessionTerminal.tsx:112`	1 h	**Regression-Risiko — Hypothese testen!**
T12	WebGL-Renderer `@xterm/addon-webgl` — reine Performance	`SessionTerminal.tsx:76`	30 min	niedrig, GL-Context-Loss im WebView2 beachten
T13	Strukturiertes Session-Log parallel zum xterm-Buffer (eigene Persistenz)	neue Module	~1 Tag	niedrig — additive

FIX-OPTIONEN

Option A — Conservative (1 Tag)

T1 + T2 + T3 + T8

- Env-Var `CLAUDE_CODE_NO_FLICKER=0` (Rust)
- `windowsPty` + `scrollOnUserInput: false` + `convertEol: true` (TS)
- Total: ~40 min Code, ~2–3 h Testing

→ Erwartete Wirkung: 70 % der „ständigen Probleme" weg, weil Layer 4 (Claude) gezähmt + Layer 5 (Config) behoben.

Option B — Root-Cause (3 Tage)

Option A + T4 + T5 + T6 + T7 + T11

- UTF-8-Carry + Font-Order + Resize-Guard + Serialize + 150ms-Teardown-Prüfung
- Total: ~6–8 h Code, ~1 Tag Testing

→ Erwartete Wirkung: Alle 10 Hypothesen adressiert, außer H4 (async Detector) und H7 (dyn. PTY).

REGRESSIONS-RISIKEN

Fix	Risiko	Gegen-Maßnahme
`CLAUDE_CODE_NO_FLICKER=0`	Claude könnte die Var deprekieren / umbenennen	Env-Var nur **zusätzlich** setzen, nicht ersetzen; Feature-Flag in App-Settings
`scrollOnUserInput: false`	Tippen scrollt nicht mehr automatisch	Manuell in `onData` triggern wenn `!userScrolledUpRef`
UTF-8-Carry-Buffer	Edge-Case: sehr langes invalides UTF-8 → `byte_carry` wächst unbounded	Cap bei 8 Bytes (max UTF-8 = 4 Bytes, 8 ist safe)
`term.open()` nach fonts.ready	`fonts.ready` hängt ohne Custom-Font → Terminal öffnet nie	Timeout auf `Promise.race([fonts.ready, sleep(500)])`
`scrollTrackTimer` entfernen	Regression falls d12f0ec + 8b820f5 nicht vollständig fixt	Regression-Test + A/B-Build
`windowsPty`	Option ist in xterm v6 stabil; Build-Number-Check meistens no-op	Fallback auf `{ backend: 'conpty' }` ohne buildNumber

OFFENE FRAGEN (Phase 2 Blocker)

1. **Nutzt Claude-CLI wirklich Alt-Screen bei `--dangerously-skip-permissions`?**
   → Verify via `script -e /tmp/log claude` oder `strace` (Linux) / ConHost-Trace (Windows)

2. **Ist `CLAUDE_CODE_NO_FLICKER` offiziell dokumentiert oder interne Escape-Hatch?**
   → WebFetch auf Anthropic-Docs; fallback: Env-Flag nur als User-opt-in via Settings-Toggle

3. **Post-d12f0ec: ist der 150ms `scrollTrackTimer` redundant?**
   → T11: entfernen, alle Scroll-Tests laufen lassen, A/B-UX-Check

4. **Bricht Frontend-State wirklich durch UTF-8-Split?**
   → T4-Prep: Fuzz-Test mit gezielter 4095-Byte-Emoji-Platzierung

5. **Welche Claude-Code-Version ist aktuell im Test-Umfeld?**
   → v2.1.89 oder später? Bestimmt ob `NO_FLICKER` greift

WAS WIR NICHT TUN SOLLTEN (Phase 2)

> **Neue Symptom-Guards hinzufügen.** Wenn wir `userScrolledUpRef` noch einmal neu verdrahten, ohne Layer 1/4 zu adressieren, bauen wir Patch #7 auf den selben Stack.

Red Flags aus der Archäologie:

- Drei Commits in 10 Tagen die alle Auto-Scroll-Logic touchen = Symptom-Jagd
- „150ms Delay" ist ein Code-Smell — es gibt selten einen echten Grund für genau 150 ms

Regel für Phase 2:

- Jeder Fix muss an mindestens **eine konkrete Hypothese (H1–H10)** oder **Layer (1–5)** gebunden sein
- Jeder Fix bekommt Regression-Test
- Kein neuer `setTimeout` ohne begründbaren Zweck

PHASE-2-VORSCHLAG

Empfehlung: Option A (Conservative) zuerst.

Begründung:

- Minimal-invasiv, <50 Zeilen Code
- 2 der 4 Fixes sind Config-One-Liner
- Env-Var-Fix adressiert die **wahrscheinlichste** Einzel-Ursache (H2, Claude-TUI-Protokoll)
- Falls User danach immer noch Probleme meldet → Phase-2-Iteration mit T4–T7

Reihenfolge für Implementation (F1 → F2 → F3):

1. **F1:** T1 (env var) + T2 (windowsPty) + T8 (convertEol) — atomic commit `fix(sessions): reduce claude-tui scroll pathologies`
2. **F2:** T3 (scrollOnUserInput manual) — separate commit `fix(sessions): decouple auto-scroll from user-input`
3. **F3:** Regression-Tests für alle 3 — separate commit `test(sessions): regression coverage for scroll-preservation`
4. **F4:** QA-Gate (tsc, lint, cargo check, tests), local Production-Build
5. **STOP** — User-Verifikation im Build

WARTEPUNKT — User-Freigabe nötig

Phase 1 abgeschlossen:

- 4 Agenten deployed, Findings synthesiert
- 10 Hypothesen gerankt, 13 Experimente testbar
- 2 Fix-Optionen (Conservative / Root-Cause)

Warte auf:

- ✅ **„Go Option A"** — Phase 2 starten mit minimalinvasivem Fix
- 🟡 **„Go Option B"** — ambitionierter Fix mit UTF-8 + Serialize
- 🔄 **„Erst Experimente"** — Test-Matrix einzeln laufen, dann entscheiden
- ❌ **„Feedback"** — Analyse überarbeiten, spezifische Layer vertiefen

Dichte-Quelle: `reports/session-scroll-multi-agent-analyse-v1.6.26.{md,html,pdf,pptx}`

Keine Code-Änderungen ohne Freigabe — bug-fix-pipeline Phase-Gate.

ANHANG · Quell-Issues (extern)

- claude-code #41965 — v2.1.89 flicker-free zerstört Scrollback
- claude-code #37389 — Viewport snapt in Scrollback-Top
- claude-code #13637 — Progressbar/Carriage-Return-Handling
- claude-code #20094 — Resize langsam in v2.1.15
- xterm.js #2666 — ConPTY Wrap-Detection
- xterm.js #2798 — PowerShell Resize
- xterm.js #3513 — Scrollback-Handling bei Resize
- xterm.js Encoding Guide

ANHANG · Interne Code-Referenzen

Datei	Zeile	Thema
`src-tauri/src/session/manager.rs`	92–98	Initial PTY-Size 24×80
`src-tauri/src/session/manager.rs`	179–190	UTF-8-lossy Reader
`src-tauri/src/session/manager.rs`	240–285	Synchroner agent_detector
`src-tauri/src/session/manager.rs`	449–453	PowerShell -NoExit -Command
`src/components/sessions/SessionTerminal.tsx`	62–80	xterm-Config
`src/components/sessions/SessionTerminal.tsx`	93–107	Ctrl+C-Handler
`src/components/sessions/SessionTerminal.tsx`	112–121	150ms scrollTrackTimer
`src/components/sessions/SessionTerminal.tsx`	124–128	onData → write_session
`src/components/sessions/SessionTerminal.tsx`	138–143	term.write Callback mit isAtBottom
`src/components/sessions/SessionTerminal.tsx`	165–173	debouncedFit + ResizeObserver
`src/components/sessions/SessionTerminal.tsx`	176–180	document.fonts.ready.then(runFit)
`src/components/sessions/SessionManagerView.tsx`	—	Always-Mounted + Unified Grid

ENDE PHASE 1

Status: Analyse fertig. Marp-Deck in `reports/`.
Nächster Schritt: User-Entscheidung.

> „Nicht noch eine Schicht Guards. Die Root-Causes sitzen in Layer 1, 3, 4, 5 — Layer 6 ist gelöst."

— Bug-Fix-Pipeline Phase 1 Gate · warte auf Freigabe.